



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA
E REGGIO EMILIA

Dipartimento di Ingegneria "Enzo Ferrari"
Corso di Laurea Triennale in Ingegneria Informatica (L-08)

Traffic Shaping in Userspace: Fattibilità e Limiti

Tutor:
Prof. Riccardo Lancellotti

Candidato:
Leonardo Donati
Matricola 191271

Indice

1	Introduzione	3
2	Traffic Shaping	4
2.1	Definizione	4
2.2	Traffic Shaping in Ambienti Linux	4
2.2.1	Queueing Discipline (qdisc)	4
2.2.2	Token Bucket Filter (TBF)	5
2.2.3	Hierarchical Token Bucket (HTB)	6
2.3	Esempi Reali di Utilizzo	8
2.3.1	Limitazione Banda su Backup	8
2.3.2	Server con Multipli Servizi	8
3	Kathará	10
3.1	Introduzione	10
3.2	Architettura di Kathará	10
3.3	Configurazione	11
3.3.1	Riproduzione dello scenario HTB in Kathará	12
3.4	Dipendenza dai moduli del kernel host	14

Capitolo 1

Introduzione

Capitolo 2

Traffic Shaping

2.1 Definizione

DEFINITION Traffic Shaping

Il traffic shaping è una tecnica di gestione della larghezza di banda utilizzata nelle reti informatiche, che ritarda alcuni o tutti i datagrammi IP per adeguarli al profilo di traffico desiderato. Consente di migliorare le prestazioni della rete controllando in il traffico in entrata e in uscita o di simulare condizioni di reti reali.

A differenza di tecniche basate sullo scarto di datagrammi IP in eccesso, il traffic shaping ne ritarda l'invio, trattendoli in una coda.

Esistono alcuni scenari tipici in cui è applicato:

- **Limitazione** della banda per utenti e servizi in una rete;
- **Garanzia** di banda minima per determinati flussi;
- **Prioritizzazione** del traffico;
- **Minimizzazione** della latenza;

2.2 Traffic Shaping in Ambienti Linux

In ambienti Linux il traffic shaping può essere effettuato con il comando `tc` (traffic control) [1], disponibile nella suite di comandi `iproute2` [2].

Nella pagina ufficiale del manuale di [1], viene evidenziato come l'elaborazione del traffico è gestita tramite tre tipi di oggetti: **qdisc**, **classi** e **filtri**.

2.2.1 Queueing Discipline (qdisc)

DEFINITION qdisc

qdisc è l'abbreviazione di queueing discipline: rappresenta il meccanismo con cui il kernel gestisce l'invio dei pacchetti su un'interfaccia, accodandoli secondo una politica configurabile [1].

Alcune qdisc possono essere **classful**, quindi contenere classi con con a loro volta una propria qdisc, creando di fatto una gerarchia. Il kernel può decidere di inviare un pacchetto di una classe invece che un'altra, per dare priorità a determinati tipi di traffico cercando di estrarre pacchetti prima da certe classi.

Eventualmente, le qdisc possono essere pure **classless**, potendo solo riordinare, ritardare o scartare pacchetti.

2.2.2 Token Bucket Filter (TBF)

DEFINITION Token Bucket Filter

Disciplina di accodamento **classless** disponibile per il controllo del traffico nel comando **tc**. Funziona puramente da shaper e non si occupa mai dello scheduling del traffico. [3]

TBF è molto leggero e non appesantisce la rete né il processore. Il suo unico limite è che può lavorare su una interfaccia usando una sola coda.

Algoritmo

L'implementazione consiste in un buffer (**bucket**) che viene costantemente riempito di gettoni (**tokens**), a una velocità specifica **rate**. Il bucket ha una capienza (**burst**), che indica quanti gettoni può contenere al massimo. Un token equivale a circa un byte.

Quando un pacchetto deve essere inviato, i gettoni necessari per inviarlo vengono rimossi dal bucket. Nel caso non fossero sufficienti, il pacchetto viene inserito in una coda finché non vengono generati abbastanza gettoni per inviarlo, in caso contrario viene mandato immediatamente. Sfruttando il rate dei gettoni e il burst è possibile, quindi, ritardare l'invio dei pacchetti.

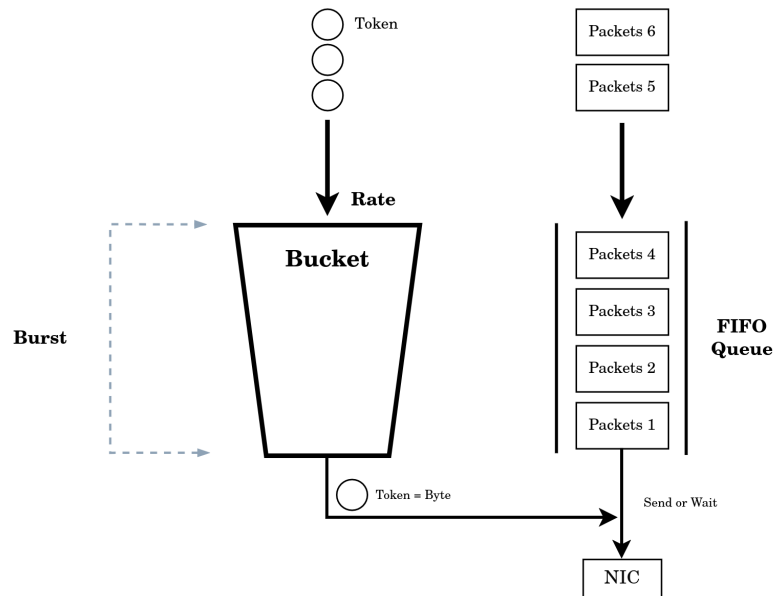


Figura 2.1: Funzionamento di TBF

La coda dei pacchetti utilizza una politica **FIFO** (First In, First Out) e la sua grandezza in byte può essere decisa tramite il parametro **limit**. Se piena, i pacchetti successivi vengono scartati direttamente.

Parametri comando tc per TBF [3]		
Parametro	Unità di Misura	Descrizione
rate	Mbit/s	Velocità di generazione dei tokens.
burst	Byte	Capienza del bucket.
limit	Byte	Capienza della coda.

2.2.3 Hierarchical Token Bucket (HTB)

DEFINITION Hierarchical Token Bucket

Disciplina di accodamento **classful** disponibile per il controllo del traffico nel comando `tc`. Utilizzato per fare traffic shaping controllando la banda in uscita di un'interfaccia, permettendo di dividere vari flussi tramite l'uso di un albero per creare una **gerarchia di classi**. L'algoritmo utilizzato per l'elaborazione del traffico è token bucket filter [4].

Ogni classe rappresenta un nodo di un **albero** e, ad eccezione della **radice**, ha una classe **genitore**. Questa struttura permette di suddividere ricorsivamente la banda assegnata a una classe tra le sue classi figlie, ciascuna configurabile con i propri parametri: **rate** e **ceil**.

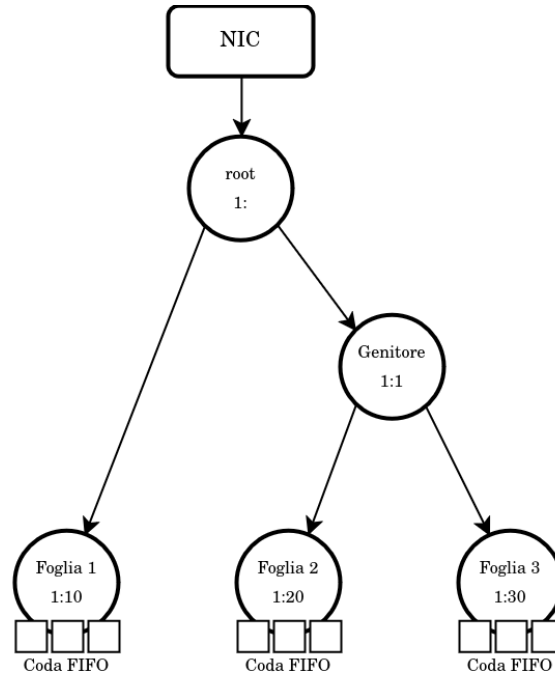


Figura 2.2: Gerarchia HTB

Le code FIFO di pacchetti in attesa possono essere solo sulle foglie dell'albero, i nodi intermedi non possono accodare.

Algoritmo di Condivisione della Banda

Tramite questo algoritmo, quando una classe necessita di più banda di quella garantita dal proprio rate, può prendere in prestito la capacità non utilizzata dalle classi che condividono lo stesso genitore, fino al limite imposto dal proprio ceil. Il parametro rate indica la banda **minima** garantita, invece il parametro ceil indica la banda **massima** utilizzabile.

Una classe genitore conosce la somma della banda effettivamente utilizzata dalle proprie classi figlie e può quindi ridistribuirle a quelle che ne richiedono di più. Considerando che come algoritmo viene usato token bucket, possiamo includere tutti i casi possibili dei prestiti in tre situazioni diverse [5]:

- Il numero di tokens di una classe è sufficiente per inviare immediatamente il pacchetto.
- Il numero di tokens di una classe non è sufficiente per inviare immediatamente un pacchetto, ma è possibile chiedere un prestito alla classe genitore, visto che la banda non è saturata.
- Il numero di tokens di una classe non è sufficiente per inviare immediatamente un pacchetto e non è possibile prendere in prestito ulteriore banda. Il pacchetto viene aggiunto alla coda finché non sarà possibile spedirlo.

I parametri utili per l'algoritmo di prestito sono:

Parametro	Unità di Misura	Descrizione
rate	Mbit/s	Velocità minima garantita di generazione dei tokens.
burst	Byte	Capienza del bucket.
ceil	Mbit/s	Velocità massima possibile di generazione dei tokens.
cburst	Byte	Capienza del bucket necessaria quando la generazione dei tokens è al massimo possibile.
quantum	Byte	Byte da inviare prima di passare a un'altra classe.

Filtri

DEFINITION Filtro

Metodo usato per inserire il pacchetto in una determinata classe in base a diverse caratteristiche, è possibile farlo pure tramite il comando `tc` (traffic control) [1]. Non è l'unico modo pensato per classificare i pacchetti, possono essere usate pure le iptables [6].

I filtri sono l'oggetto che lega le code di pacchetti alle classi e possono farlo secondo vari criteri: ip, porte, protocolli, fwmark nel caso vengano usate le iptables e molti altri.

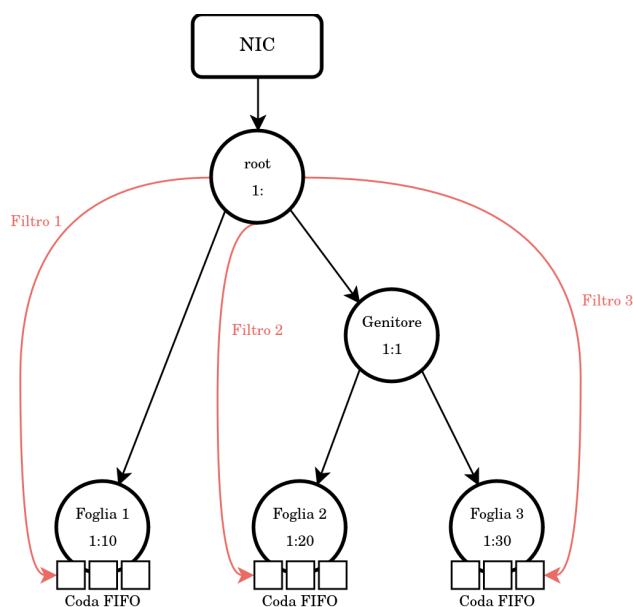


Figura 2.3: Classificazione pacchetti con filtri in HTB.

Normalmente la classificazione avviene una sola volta, con i filtri applicati al nodo radice, ma è possibile aggiungere ulteriori filtri nei nodi intermedi.

2.3 Esempi Reali di Utilizzo

Il traffic shaping è usato anche in contesti reali. Nei prossimi paragrafi, vorrei mostrare alcuni casi d'uso in cui è utile applicare discipline di accodamento come Token Bucket Filter o Hierarchical Token Bucket.

2.3.1 Limitazione Banda su Backup

SCENARIO

Un piccolo ufficio ha una connessione internet condivisa da 20 Mbit/s in upload. Un server esegue un backup automatico verso il cloud in diversi orari della giornata e se lasciato libero satura completamente la banda in upload. Si vuole limitare la velocità del traffico in uscita in modo da permettere ai dipendenti di lavorare tranquillamente.

Server Backup

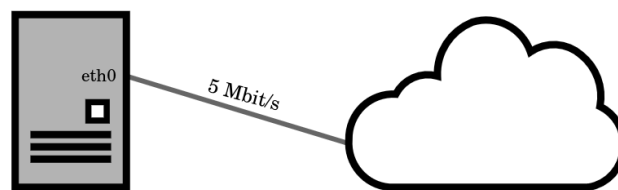


Figura 2.4: Rappresentazione grafica del primo scenario.

In questo caso, è possibile limitare il traffico sull'interfaccia di rete del server di backup. Ipotizziamo che l'interfaccia sia `eth0` e che si voglia limitare il traffico a 5 Mbit/s:

BASH Comando tc

```
tc qdisc add dev eth0 root tbf rate 5mbit burst 32kb limit 100kb
```

Ora, la connessione è correttamente limitata.

2.3.2 Server con Multipli Servizi

SCENARIO

Un server di un'azienda ospita tre servizi per tre clienti diversi: un servizio streaming, un sito web con molto traffico e un servizio di file hosting.

La banda totale disponibile e garantita è 100 Mbit/s. Si vuole garantire più banda al cliente del sito web rispetto agli altri due clienti. Gli altri due servizi avranno un minimo di banda garantita.

Server

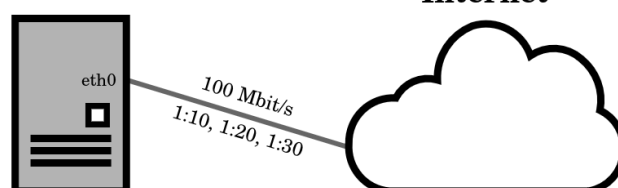


Figura 2.5: Rappresentazione grafica del secondo scenario.

In questo caso, con opportuni filtri è possibile inserire i pacchetti nelle code delle classi corrette: 1:10 per il sito web, 1:20 per lo streaming e 1:30 per il file hosting.

BASH Classi e parametri con comando tc

```
# Nodo root e genitore
tc qdisc add dev eth0 root handle 1: htb default 30
tc class add dev eth0 parent 1: classid 1:1 htb rate 100mbit ceil 100mbit

# Classe sito web
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 50mbit ceil 100mbit

# Classe servizio streaming
tc class add dev eth0 parent 1:1 classid 1:20 htb rate 35mbit ceil 60mbit

# Classe servizio file hosting
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 15mbit ceil 20mbit
```

Il nodo genitore ha un minimo e un massimo di banda di 100Mbit/s, che dividerà fra le tre classi, permettendo pure il prestito nel caso fosse possibile.

L'ultima cosa che rimane da fare è scrivere i vari filtri che divideranno il traffico nelle opportune code.

BASH Filtri con comando tc

```
# Filtri per sito web
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 80 0xffff
↪ flowid 1:10
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 443
↪ 0xffff flowid 1:10

# Filtro per servizio streaming
tc filter add dev eth0 protocol ip parent 1:0 prio 2 u32 match ip sport <port>
↪ 0xffff flowid 1:20

# Filtro per servizio file hosting
tc filter add dev eth0 protocol ip parent 1:0 prio 3 u32 match ip sport <port>
↪ 0xffff flowid 1:30
```

In questo modo, il traffico in uscita con porta sorgente 80 e porta 443 (http e https), andrà correttamente nella classe creata per il sito web. I pacchetti degli altri due servizi verranno classificati, come fatto in precedenza, in base alla loro porta sorgente.

Nel comando è stata inserita una diversa priorità per la valutazione dei filtri.

Capitolo 3

Kathará

3.1 Introduzione

DEFINITION Kathará

Kathará è un framework open source basato su container Docker che permette di creare, configurare ed eseguire reti virtuali, integrando in un unico ambiente tecnologie come **SDN** (Software Defined Network) e **NFV** (Network Function Virtualization) [7].

Ogni nodo di rete è simulato con un container. Il comportamento del nodo dipende dall'immagine usata, permettendo l'uso di protocolli tradizionali (OSPF, BGP, ecc), SDN o servizi applicativi (DNS, Apache, ecc). I vari dispositivi comunicano tra loro attraverso LAN virtuali di livello 2, riproducendo il comportamento di un collegamento fisico.

La scelta di basarsi su container invece che su macchine virtuali, rende Kathará uno strumento molto leggero. Ogni container richiede risorse nettamente inferiori rispetto a una macchina virtuale tradizionale, permettendo di simulare topologie complesse su hardware meno performante. Inoltre, permette di sfruttare bene tecnologie come NFV (Network Function Virtualization), che utilizza funzioni software per sostituire dispositivi hardware, e SDN (Software Defined Network), che permette di separare la logica di controllo della rete, dalla logica di invio dei pacchetti.

3.2 Architettura di Kathará

L'architettura di Kathará è composta da tre moduli principali: **Scripts**, **Operations** e il **Container Platform Adapter** [7].

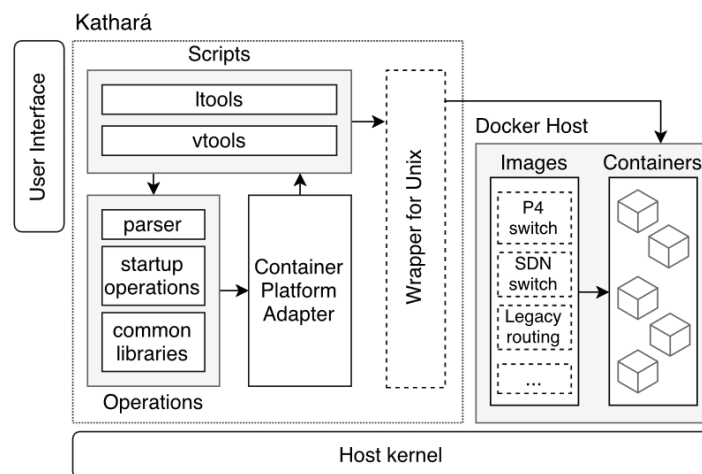


Figura 3.1: Architettura di Kathará [7]

Il modulo Scripts contiene le operazioni di base che utente può fare per avviare, fermare e mettere in pausa i nodi di rete. Si dividono in due tipologie:

1. **ltools:** valgono per tutti i nodi inseriti nel file di configurazione (**lstart**, **lclean**).
2. **vtools:** valgono per un singolo nodo (**vstart**, **vclean**).

Il modulo Operations, invece, racchiude alcune funzionalità che servono a Kathará per funzionare, come:

- **Parser:** traduce i file di configurazione o i parametri passati da riga di comando in nodi della rete con eventuali collegamenti. Si occupa anche di tradurre ulteriori opzioni, come risorse assegnate a ogni nodo o immagini Docker da utilizzare.
- **Start-Up Operations:** serve a garantire compatibilità fra i diversi sistemi.
- **Common Libraries:** genera una serie di operazioni indipendenti da eseguire.

Infine, il modulo del Container Platform Adapter converte i comandi generati in precedenza in direttive Docker (o altre piattaforme) per creare e collegare fra di loro i container. L'output dell'adapter ritorna al modulo Scripts, che lo invia al Docker daemon.

Nei sistemi Unix, per ragioni di sicurezza, i comandi devono prima essere approvati da un wrapper che scarta qualsiasi comando non ritenuto sicuro.

3.3 Configurazione

Supponiamo di avere un rete con tre nodi: due host e uno switch. Tramite Kathará è possibile crearli e avviarli tramite riga di comando, con i vtools, oppure, tramite file di configurazione, con gli ltools [7]. Questa seconda modalità è quella che userò per configurare la rete.

Il primo file da creare è quello in cui vengono definiti i nodi e se necessario altre opzioni, come le immagini dei container Docker.

KATHARÁ lab.conf

```
# Collegamento fra un host e porte dello switch
pc1[0]="A"
sw1[0]="A"

pc2[0]="B"
sw1[1]="B"

# Immagine per lo switch
sw1[image]="kathara/quagga"
```

Successivamente, è possibile creare e avviare i container con il comando **kathara lstart**, partendo dalle immagini di default o da quelle specificate. Kathará, inoltre, aprirà per ogni nodo un terminale, in modo da poter fare modifiche o test.

In alternativa, è possibile creare dei file di configurazione per ogni nodo, scrivendo al suo interno una lista di comandi da far fare in automatico all'accensione del container. Per l'host **pc1**, nel suo file di configurazione si può assegnare un ip e accendere la rispettiva interfaccia.

KATHARÁ pc1.startup

```
# Assegnare un ip
ip addr add 192.168.1.1/24 dev eth0

# Accendere l'interfaccia eth0
ip link set dev eth0 up
```

Tramite questa modalità è possibile configurare pure il secondo host e lo switch. Una volta digitato il comando **kathara lstart**, tutti i file di configurazione verranno usati per creare la rete.

3.3.1 Riproduzione dello scenario HTB in Kathará

Sfruttando Kathará, si possono riproporre gli esempi del capitolo precedente sull'utilizzo delle qdisc tramite il comando `tc`, testandolo con diversi nodi. In particolare, lo scenario su HTB è utile per dimostrare che è possibile fare traffic shaping con questo framework. Facendo

riferimento allo scenario dell'esercizio 2.3.2, il primo file da creare sarà `lab.conf`, in cui dovranno essere definiti i tre clienti e il server con i servizi.

KATHARÁ lab.conf

```
# Clienti
c1[0] = "A"
c2[0] = "A"
c3[0] = "A"

# Server
server[0] = "A"
```

Specificando per ognuno il dominio "A", Kathará simulerà il comportamento di una rete locale con uno switch che collega tutti i nodi. Successivamente, per ogni host, dovrà essere creato un file che conterrà tutti i comandi necessari per poter comunicare fra di loro e nel caso del server pure quelli utili per il traffic shaping.

KATHARÁ c1.startup

```
# Assegnazione ip
ip addr add 10.0.0.1/24 dev eth0

# Accensione interfaccia eth0
ip link set dev eth0 up
```

Questa configurazione sarà uguale anche per gli altri clienti, ad eccezione dell'indirizzo ip differente. Invece, in `server.startup`, verranno inseriti pure i comandi con `tc`.

BASH server.startup

```
# Assegnazione ip
ip addr add 10.0.0.10/24 dev eth0

# Accensione interfaccia eth0
ip link set dev eth0 up

# Nodo root e genitore
tc qdisc add dev eth0 root handle 1: htb default 30
tc class add dev eth0 parent 1: classid 1:1 htb rate 100mbit ceil 100mbit

# Classe sito web
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 50mbit ceil 100mbit

# Classe servizio streaming
tc class add dev eth0 parent 1:1 classid 1:20 htb rate 35mbit ceil 60mbit

# Classe servizio file hosting
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 15mbit ceil 20mbit
```

Infine, all'interno di quest'ultimo file, o sul container una volta avviato, dovranno essere creati pure i filtri per classificare i flussi.

BASH `server.startup`

```
# Filtri per sito web
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 80 0xffff
↪ flowid 1:10
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 443
↪ 0xffff flowid 1:10

# Filtro per servizio streaming
tc filter add dev eth0 protocol ip parent 1:0 prio 2 u32 match ip sport 554
↪ 0xffff flowid 1:20

# Filtro per servizio file hosting
tc filter add dev eth0 protocol ip parent 1:0 prio 3 u32 match ip sport 21 0xffff
↪ flowid 1:30
```

Risultati

Per capire gli effetti che il traffic shaping ha portato alla rete, è utile confrontare i tempi di invio di dati con o senza HTB. La formula per calcolare i tempi di invio ideali è:

$$t = \frac{\text{dim} \times 8}{\text{rate}}$$

Sul nodo del server è stato creato un file di 100 MB e poi è stato posto in ascolto sulle tre porte: 443, 554 e 21. Ogni client ha scaricato contemporaneamente il file, tramite il comando `nc`, collegandosi alla porta del proprio servizio.

Per ciascun trasferimento è stato misurato il tempo reale.

Classe	Tempo Ideale	Tempo Reale
1:10	~ 16s	19.20s
1:20	~ 23s	23.33s
1:30	~ 53s	48.47s

Nell'esempio, si può notare come la classe 1:30 abbia impiegato meno tempo di quello ideale. Questo è dovuto al fatto che, dopo che il primo cliente ha finito di leggere i 100MB di file, le altre due classi si sono divise la banda tramite l'algoritmo di prestito.

Inoltre, per capire l'effetto del traffic shaping nella rete, è utile guardare anche i risultati senza l'utilizzo di HTB.

Classe	Tempo
Sito Web	22.81s
Streaming	23.04s
File Hosting	24.64s

Si può notare come i tempi, simili tra loro, rappresentino un terzo della banda totale, ossia circa 33Mbit/s per ciascun nodo, dato che $\frac{100\text{MB} \times 8}{33\text{Mbit/s}} = 24\text{s}$.

In conclusione, applicando una disciplina di accodamento come HTB per fare traffic shaping, ha permesso al servizio del sito web di avere a disposizione una banda maggiore, terminando il trasferimento per primo. Invece, ha rallentato il servizio di file hosting, dedicandogli una banda minore.

L'utilizzo di Kathará per creare la rete, ha permesso di calcolare i tempi reali in una LAN nelle diverse modalità proposte.

Servizio	Tempo con HTB	Tempo senza HTB
Sito Web	19.20s	22.81s
Streaming	23.33s	23.04s
File Hosting	48.47s	24.64s

3.4 Dipendenza dai moduli del kernel host

L'uso di container Docker rende la simulazione leggera e flessibile, permettendo di scegliere per ogni nodo un'immagine diversa, ma crea un limite importante. A differenza delle macchine virtuali, i container non eseguono un kernel proprio, ma condividono quello dell'host su cui gira Docker, ciò che un nodo può fare a livello di rete dipende dai moduli caricati nel kernel dell'host.

Il comando `tc`, eseguito dentro un container, invia solamente richieste al kernel, non implementa effettivamente le discipline di accodamento. Le qdisc, le classi e i filtri sono realizzati da diversi moduli del kernel:

Modulo	Descrizione
<code>sch_tbf</code>	Modulo per qdisc TBF
<code>sch_htb</code>	Modulo per qdisc HTB
<code>cls_u32</code>	Modulo per filtri u32

Bibliografia

- [1] Alexey N. Kuznetsov. *tc* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [2] Alexey Kuznetsov. *iproute2* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/ip-route.8.html>.
- [3] Alexey N. Kuznetsov. *tc-tbf* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/tc-tbf.8.html>.
- [4] Martin Devera. *tc-htb* — *Linux manual page*. 2001. URL: <https://man7.org/linux/man-pages/man8/tc-htb.8.html>.
- [5] Martin Devera. *tc-htb* — *Htb official theory page*. 2001. URL: <http://luxik.cdi.cz/~devik/qos/htb/manual/theory.htm>.
- [6] Michael Neuling Rusty Russell. *iptables* — *Linux manual page*. 2001. URL: <https://man7.org/linux/man-pages/man8/iptables.8.html>.
- [7] Gaetano Bonofiglio et al. «Kathará: A container-based framework for implementing network function virtualization and software defined networks». In: *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*. 2018, pp. 1–9. DOI: 10.1109/NOMS.2018.8406267.